Neural Network for Handwritten Digit Recognition Using MNIST

Deep Learning CNN design using MNIST

Key points

Dataset: MNIST

Data Preparation: Getting the data ready for training.

Model Selection: Why choose a Convolutional Neural Network?

Building & Training: Creating and teaching the model

Performance Evaluation: How well did our model do?

Method Comparison: How does this design compare to other methods?

Conclusion & Future Work: Key takeaways and Summary

The Dataset: MNIST

A classic computer vision dataset, often called “Hello World” of ML

70,000 greyscale 28x28 pixel images.

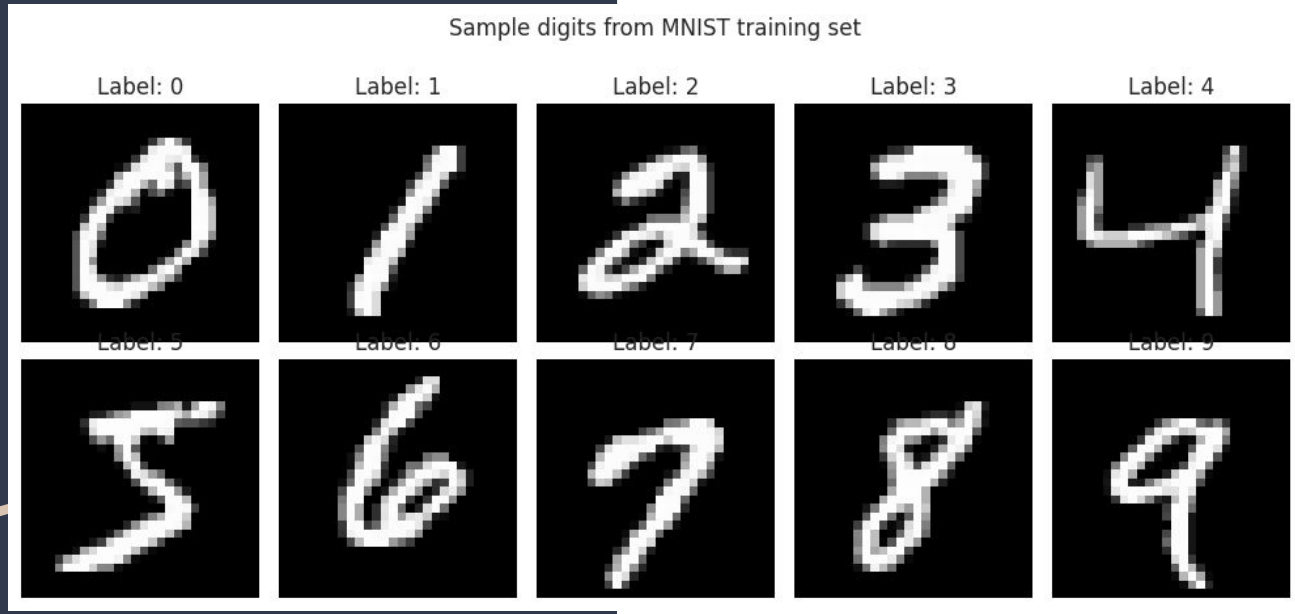
10 Classes: Digits 0 through 9

Pre split: 60,000 training + 10,000 test

The Goal: Train a model to correctly label a handwritten digit it has never seen before

Sample digits

One example per class. Real handwriting varies in style, stroke and slant.



Data Preparation: Loading & Splitting

MNIST comes pre-split: 60,000 train + 10,000 test

We split 60,000 train set further into 80/20

- **Training Set (80%):** 48,000 images that the model is learning from
- **Validation Set (10%):** 12,000 images to prevent models from overfitting (memorizing the set)
- **Test Set (10%):** 10,000 images, final test on data the model has never seem prior

Data Preparation: Normalization

Scaling pixel values from their original range (0-255) to a new range (0-1).

Neural networks learn much faster and more reliably when their input numbers are small. It helps the training process run smoothly.

We use a division : `image / 255.0`

Model Selection: What makes a CNN a good choice?

CNN learns features automatically.

- Early layers learn simple shapes like edges and curves.
- Deeper layers combine to learn more complex features

.

Model Architecture

- We'll use a simple, sequential CNN architecture.
- It's a stack of layers that progressively learns more complex features.
  1. **Convolutional Layer:** 32 filters of size 3x3 (input shape 28x28x1)
  2. **Pooling Layer:** Shrinks the image down. (2x2)
  3. **Convolutional Layer:** 64 filters of size 3x3
  4. **Pooling Layer:** Shrinks it again.
  5. **Flatten Layer:** Prepares data for the final classifier.
  6. **Dense Layer (64 units):** Output layer (10 units, softmax)

Activation Functions

- Activation functions decide which signal each neuron passes on
- ReLU (hidden layers): outputs max(0,x). Simple and fast, avoids vanishing gradient problem.
- Softmax (output layer) : converts raw scores into a probability distribution over 10 digit classes that sums to 1
- Softmax pairs naturally with cross-entropy loss

Model Compilation

Before training, we need to configure the learning process.

1. **Optimizer:** `Adam`
   - The "engine" that updates the model to make it better. Adam is an efficient, all-purpose choice.
2. **Loss Function:** `SparseCategoricalCrossentropy`
   - This measures how wrong the model's predictions are. The goal of training is to minimize this loss.
3. **Metrics:** `accuracy`
   - The metric we want to watch during training.

Model Training

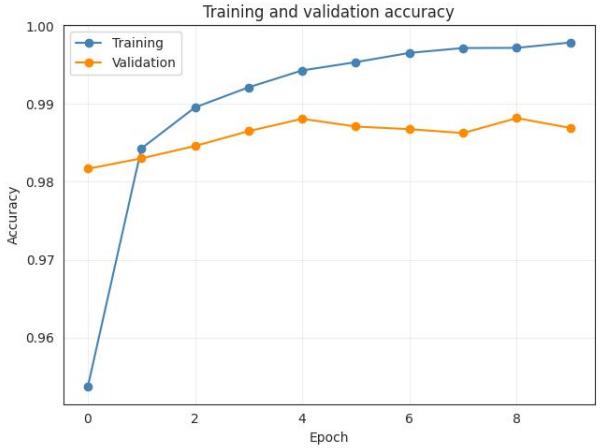
- Model is being fit to our training data.
- MNIST builds quickly, by epoch 5 validation accuracy is already close to it’s final value.
- **Epochs = 10:** The model will see the entire training dataset 10 times.
- **Batch Size:** The model looks at images in batches for efficiency purposes.
- We define **validation data** to check its performance on data it is not been training on.

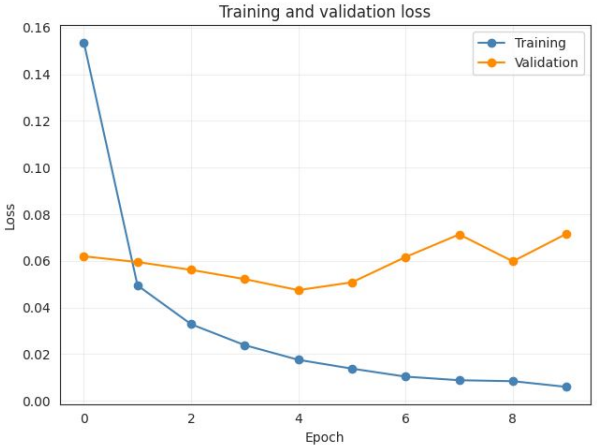
Model Performance

We can plot the training process to see how the model learned.

- **Accuracy:** Should go up for training and validation.
- **Loss:** Should go down for both.

Key Insight: After about epoch 5 validation loss began to rise slightly while training loss kept falling. Overfitting signal worth flagging





Evaluation and Testing

- We use test set - 10,000 images the model has never seen
- This gives an honest measure of generalisation
- This model achieves 99,01% accuracy (test loss: 0.0496)

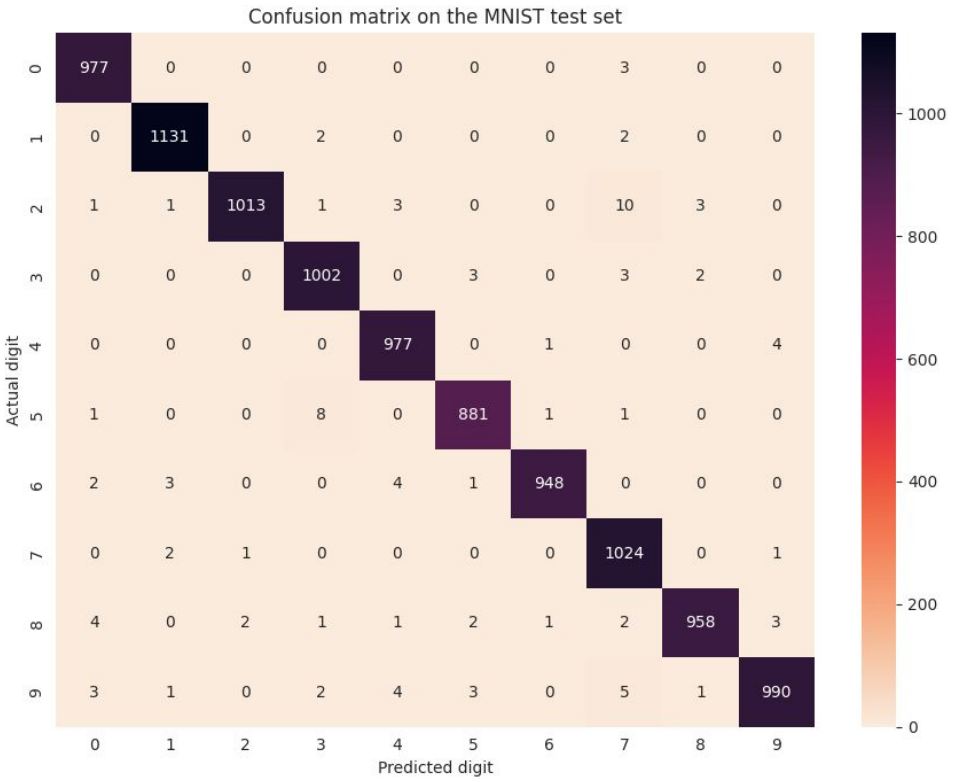
Confusion Matrix

A confusion matrix shows us where the model gets confused.

Rows: actual digit

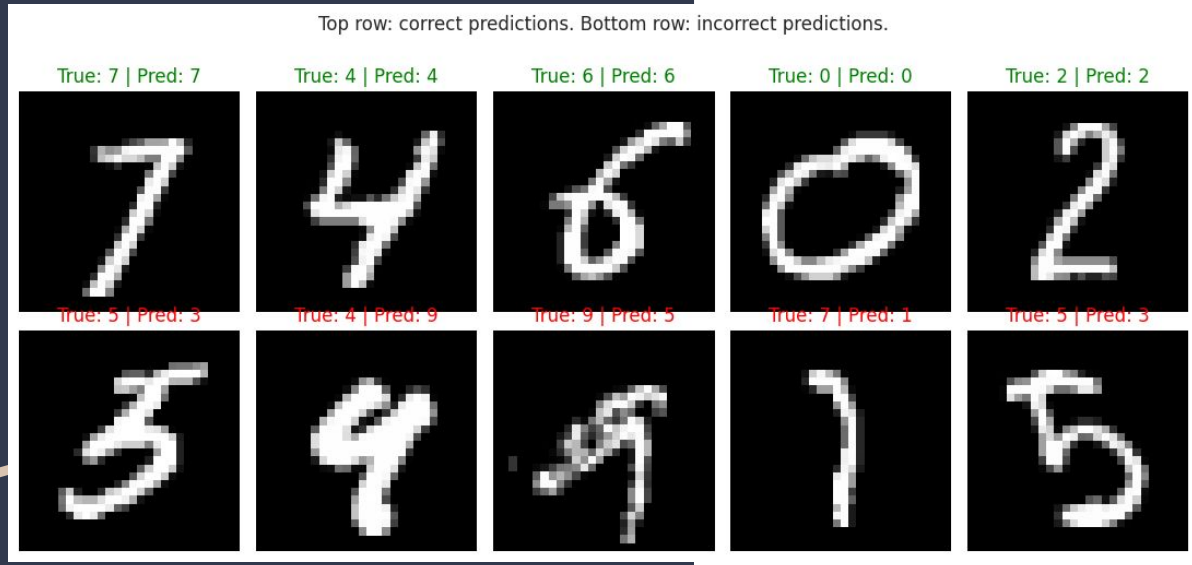
Columns: predicted digit

Off-diagonal: Mistakes are clear : we can see which digits are commonly mixed up. (2 and 7, 5 and 3, 4 and 9)



Example Predictions

- Many wrong predictions can involve ambiguous handwriting
- Suggests the model is close to the natural ceiling on the dataset



Conclusion

Conclusion: We successfully built, trained, and evaluated a CNN that can classify digits in the MNIST dataset with 99% accuracy. The project demonstrates the power of CNNs for learning features directly from images.

Lessons Learned: The importance of a proper data split (train/val/test) is critical for avoiding overfitting and trustworthy results. Even at 99% accuracy ~ 100 mistakes per 10,000 remain.

Future Improvements:

1. **Data Augmentation:** Small rotations and shifts to handle handwriting variance
2. **Regularisation:** Dropout or early stopping to address mild overfitting
3. **Harder variants**: Fashion-MNIST or EMNIST for more challenging tasks

References

LeCun, Y. et al. (1998) ‘Gradient-based learning applied to document recognition’, Proceedings of the IEEE, 86(11), pp. 2278–2324.

Nair, V. and Hinton, G. E. (2010) ‘Rectified linear units improve restricted Boltzmann machines’, Proceedings of ICML.

Kingma, D. P. and Ba, J. (2014) ‘Adam: A method for stochastic optimization’, arXiv:1412.6980.

Goodfellow, I., Bengio, Y. and Courville, A. (2016) Deep Learning. MIT